

Full Stack Development with MERN

Project Documentation format

1. Introduction

- **Project Title:**shopEZ-MERN E-commerce Application
- **Team Members:** 1.Sri Sai Venkat Barla
2.Sowmya Kodali
3.Sai Sreedevi Pulaparthi
4. Teomayee Hema Sridevi Geddada

2. Project Overview

- **Purpose:** ShopEZ is a full-stack MERN-based e-commerce web application designed to provide customers with a seamless online shopping experience. The platform enables users to browse products, search and filter items, manage shopping carts and wishlists, place orders, and track purchases. It also provides administrative functionalities for managing products, categories, users, and customer orders.

Features:

User Registration and Login

JWT-based Authentication

Product Management

Category Management

Product Search

Product Filtering

Shopping Cart

Wishlist

Buy Now

Order Management

Admin Dashboard

3. Architecture

- **Frontend:**The frontend is developed using React.js following a component-based architecture. Routing is handled using React Router DOM, while Axios is used for API communication. CSS Modules provide modular styling, and Context API manages authentication, cart, and wishlist states.

- **Backend:**

The backend follows the REST API architecture using Node.js and Express.js. Business logic is separated into controllers and services, while middleware handles authentication, authorization, and error handling.

Database:

The application uses MongoDB Community Server, a NoSQL database, to store application data. Mongoose is used as the Object Data Modeling (ODM) library for defining schemas and interacting with the database.

4. Setup Instructions

- **Prerequisites:**

Before running the application, install:

- Node.js
- npm
- MongoDB Community Server
- Git
- Visual Studio Code

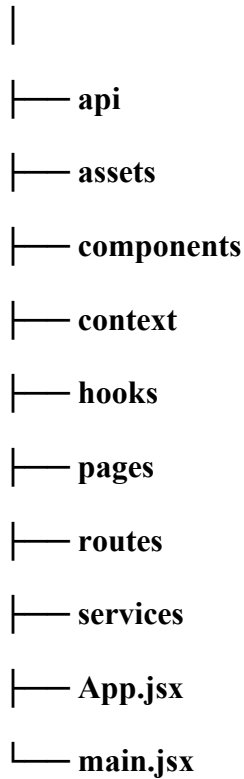
Installation:

- 1.Clone the repository
- 2.Navigate to the backend folder
- 3.Install backend dependencies
- 4.Navigate to the frontend folder
- 5.Install frontend dependencies
- 6.Create the backend `.env` file
- 7.Create frontend `.env`

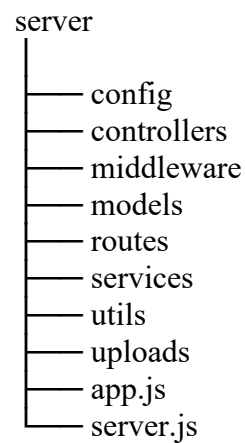
5. Folder Structure

- **Client:**

client



• Server:



6. Running the Application

- Provide commands to start the frontend and backend servers locally. ○
 - **Frontend:** `cd client,npm run dev` in the client directory.
 - **Backend:** `cd server,npm start` in server directory

7. API Documentation

 ShopEZ MERN E-Commerce Platform	API Documentation Base URL: http://localhost:5000/api Content-Type: application/json	Legend (Auth) Yes - Authentication Required No - Authentication Not Required																																																			
1. AUTHENTICATION APIS <table> <tr> <th>Method</th> <th>Endpoint</th> <th>Description</th> <th>Auth</th> </tr> <tr> <td>POST</td> <td>/auth/register</td> <td>Register a new user</td> <td></td> </tr> <tr> <td>POST</td> <td>/auth/login</td> <td>Login user and get token</td> <td></td> </tr> <tr> <td>GET</td> <td>/auth/profile</td> <td>Get logged in user profile</td> <td></td> </tr> <tr> <td>PUT</td> <td>/auth/update-profile</td> <td>Update user profile</td> <td></td> </tr> <tr> <td>POST</td> <td>/auth/logout</td> <td>Logout user</td> <td></td> </tr> </table> Example Response (Login) <pre>{ "success": true, "message": "Login successful", "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." }</pre>	Method	Endpoint	Description	Auth	POST	/auth/register	Register a new user		POST	/auth/login	Login user and get token		GET	/auth/profile	Get logged in user profile		PUT	/auth/update-profile	Update user profile		POST	/auth/logout	Logout user		2. CATEGORY APIS <table> <tr> <th>Method</th> <th>Endpoint</th> <th>Description</th> <th>Auth</th> </tr> <tr> <td>GET</td> <td>/categories</td> <td>Get all categories</td> <td></td> </tr> <tr> <td>POST</td> <td>/categories</td> <td>Create a new category (Admin)</td> <td></td> </tr> <tr> <td>PUT</td> <td>/categories/:id</td> <td>Update category by ID (Admin)</td> <td></td> </tr> <tr> <td>DELETE</td> <td>/categories/:id</td> <td>Delete category by ID (Admin)</td> <td></td> </tr> <tr> <td>GET</td> <td>/categories/:id</td> <td>Get category details by ID</td> <td></td> </tr> </table> Example Response (Get All Categories) <pre>{ "success": true, "categories": [{ "_id": "664e5c3e9a1c2b3d4e5f6781", "name": "Electronics" }, { "_id": "664e5c3e9a1c2b3d4e5f6782", "name": "Fashion" }] }</pre>	Method	Endpoint	Description	Auth	GET	/categories	Get all categories		POST	/categories	Create a new category (Admin)		PUT	/categories/:id	Update category by ID (Admin)		DELETE	/categories/:id	Delete category by ID (Admin)		GET	/categories/:id	Get category details by ID					
Method	Endpoint	Description	Auth																																																		
POST	/auth/register	Register a new user																																																			
POST	/auth/login	Login user and get token																																																			
GET	/auth/profile	Get logged in user profile																																																			
PUT	/auth/update-profile	Update user profile																																																			
POST	/auth/logout	Logout user																																																			
Method	Endpoint	Description	Auth																																																		
GET	/categories	Get all categories																																																			
POST	/categories	Create a new category (Admin)																																																			
PUT	/categories/:id	Update category by ID (Admin)																																																			
DELETE	/categories/:id	Delete category by ID (Admin)																																																			
GET	/categories/:id	Get category details by ID																																																			
3. PRODUCT APIS <table> <tr> <th>Method</th> <th>Endpoint</th> <th>Description</th> <th>Auth</th> </tr> <tr> <td>GET</td> <td>/products</td> <td>Get all products (with filters)</td> <td></td> </tr> <tr> <td>GET</td> <td>/products/:id</td> <td>Get product details by ID</td> <td></td> </tr> <tr> <td>POST</td> <td>/products</td> <td>Create a new product (Admin/Seller)</td> <td></td> </tr> <tr> <td>PUT</td> <td>/products/:id</td> <td>Update product by ID (Admin/Seller)</td> <td></td> </tr> <tr> <td>DELETE</td> <td>/products/:id</td> <td>Delete product by ID (Admin/Seller)</td> <td></td> </tr> <tr> <td>GET</td> <td>/products/search/:keyword</td> <td>Search products by keyword</td> <td></td> </tr> </table> Example Response (Get All Products) <pre>{ "success": true, "products": [{ "_id": "664e5c3e9a1c2b3d4e5f6781", "name": "Laptop", "price": 54999, "stock": 10 }, { "_id": "664e5c3e9a1c2b3d4e5f6782", "name": "Headphones", "price": 1999 }], "count": 1 }</pre>	Method	Endpoint	Description	Auth	GET	/products	Get all products (with filters)		GET	/products/:id	Get product details by ID		POST	/products	Create a new product (Admin/Seller)		PUT	/products/:id	Update product by ID (Admin/Seller)		DELETE	/products/:id	Delete product by ID (Admin/Seller)		GET	/products/search/:keyword	Search products by keyword		4. CART APIS <table> <tr> <th>Method</th> <th>Endpoint</th> <th>Description</th> <th>Auth</th> </tr> <tr> <td>GET</td> <td>/cart</td> <td>Get logged in user cart</td> <td></td> </tr> <tr> <td>POST</td> <td>/cart</td> <td>Add product to cart</td> <td></td> </tr> <tr> <td>PUT</td> <td>/cart/:id</td> <td>Update cart item quantity</td> <td></td> </tr> <tr> <td>DELETE</td> <td>/cart/:id</td> <td>Remove product from cart</td> <td></td> </tr> <tr> <td>DELETE</td> <td>/cart/clear</td> <td>Clear entire cart</td> <td></td> </tr> </table> Example Response (Get Cart) <pre>{ "success": true, "cartItems": [{ "_id": "664e5c3e9a1c2b3d4e5f6781", "product": { "name": "Laptop" }, "quantity": 1, "price": 54999 }], "total": 54999 }</pre>	Method	Endpoint	Description	Auth	GET	/cart	Get logged in user cart		POST	/cart	Add product to cart		PUT	/cart/:id	Update cart item quantity		DELETE	/cart/:id	Remove product from cart		DELETE	/cart/clear	Clear entire cart	
Method	Endpoint	Description	Auth																																																		
GET	/products	Get all products (with filters)																																																			
GET	/products/:id	Get product details by ID																																																			
POST	/products	Create a new product (Admin/Seller)																																																			
PUT	/products/:id	Update product by ID (Admin/Seller)																																																			
DELETE	/products/:id	Delete product by ID (Admin/Seller)																																																			
GET	/products/search/:keyword	Search products by keyword																																																			
Method	Endpoint	Description	Auth																																																		
GET	/cart	Get logged in user cart																																																			
POST	/cart	Add product to cart																																																			
PUT	/cart/:id	Update cart item quantity																																																			
DELETE	/cart/:id	Remove product from cart																																																			
DELETE	/cart/clear	Clear entire cart																																																			
5. WISHLIST APIS <table> <tr> <th>Method</th> <th>Endpoint</th> <th>Description</th> <th>Auth</th> </tr> <tr> <td>GET</td> <td>/wishlist</td> <td>Get logged in user wishlist</td> <td></td> </tr> <tr> <td>POST</td> <td>/wishlist</td> <td>Add product to wishlist</td> <td></td> </tr> <tr> <td>DELETE</td> <td>/wishlist/:id</td> <td>Remove product from wishlist</td> <td></td> </tr> <tr> <td>DELETE</td> <td>/wishlist/clear</td> <td>Clear entire wishlist</td> <td></td> </tr> </table> Example Response (Get Wishlist) <pre>{ "success": true, "wishlistItems": [{ "_id": "664e5c3e9a1c2b3d4e5f6781", "product": { "name": "Headphones" }, "price": 1999 }] }</pre>	Method	Endpoint	Description	Auth	GET	/wishlist	Get logged in user wishlist		POST	/wishlist	Add product to wishlist		DELETE	/wishlist/:id	Remove product from wishlist		DELETE	/wishlist/clear	Clear entire wishlist		6. ORDER APIS <table> <tr> <th>Method</th> <th>Endpoint</th> <th>Description</th> <th>Auth</th> </tr> <tr> <td>POST</td> <td>/orders</td> <td>Place a new order</td> <td></td> </tr> <tr> <td>GET</td> <td>/orders/my-orders</td> <td>Get logged in user orders</td> <td></td> </tr> <tr> <td>GET</td> <td>/orders/:id</td> <td>Get order details by ID</td> <td></td> </tr> <tr> <td>PUT</td> <td>/orders/:id/cancel</td> <td>Cancel an order</td> <td></td> </tr> <tr> <td>GET</td> <td>/orders</td> <td>Get all orders (Admin)</td> <td></td> </tr> <tr> <td>PUT</td> <td>/orders/:id/status</td> <td>Update order status (Admin)</td> <td></td> </tr> </table> Example Response (Get My Orders) <pre>{ "success": true, "orders": [{ "_id": "664e5c3e9a1c2b3d4e5f6781", "status": "Delivered", "total": 54999 }] }</pre>	Method	Endpoint	Description	Auth	POST	/orders	Place a new order		GET	/orders/my-orders	Get logged in user orders		GET	/orders/:id	Get order details by ID		PUT	/orders/:id/cancel	Cancel an order		GET	/orders	Get all orders (Admin)		PUT	/orders/:id/status	Update order status (Admin)					
Method	Endpoint	Description	Auth																																																		
GET	/wishlist	Get logged in user wishlist																																																			
POST	/wishlist	Add product to wishlist																																																			
DELETE	/wishlist/:id	Remove product from wishlist																																																			
DELETE	/wishlist/clear	Clear entire wishlist																																																			
Method	Endpoint	Description	Auth																																																		
POST	/orders	Place a new order																																																			
GET	/orders/my-orders	Get logged in user orders																																																			
GET	/orders/:id	Get order details by ID																																																			
PUT	/orders/:id/cancel	Cancel an order																																																			
GET	/orders	Get all orders (Admin)																																																			
PUT	/orders/:id/status	Update order status (Admin)																																																			
7. PAYMENT APIS <table> <tr> <th>Method</th> <th>Endpoint</th> <th>Description</th> <th>Auth</th> </tr> <tr> <td>POST</td> <td>/payments</td> <td>Create a payment for order</td> <td></td> </tr> <tr> <td>GET</td> <td>/payments/:id</td> <td>Get payment details by ID</td> <td></td> </tr> </table> Example Response (Payment) <pre>{ "success": true, "payment": { "_id": "664e5c3e9a1c2b3d4e5f6781", "method": "COD", "status": "Success", "amount": 54999 } }</pre> NOTE: All Authenticated APIs require a valid JWT token in the Authorization header. Header Format: Authorization: Bearer <your_token>	Method	Endpoint	Description	Auth	POST	/payments	Create a payment for order		GET	/payments/:id	Get payment details by ID		8. ADMIN APIS <table> <tr> <th>Method</th> <th>Endpoint</th> <th>Description</th> <th>Auth</th> </tr> <tr> <td>GET</td> <td>/users</td> <td>Get all users (Admin)</td> <td></td> </tr> <tr> <td>PUT</td> <td>/users/:id/role</td> <td>Update user role (Admin)</td> <td></td> </tr> <tr> <td>PUT</td> <td>/orders/:id/status</td> <td>Update order status (Admin)</td> <td></td> </tr> <tr> <td>DELETE</td> <td>/users/:id</td> <td>Delete user (Admin)</td> <td></td> </tr> </table> Example Response (Get All Users) <pre>{ "success": true, "users": [{ "_id": "664e5c3e9a1c2b3d4e5f6781", "name": "John Doe", "email": "john@example.com", "role": "customer" }] }</pre> Security: Passwords are hashed using bcrypt, protected routes, role-based access control, and input validation are implemented.	Method	Endpoint	Description	Auth	GET	/users	Get all users (Admin)		PUT	/users/:id/role	Update user role (Admin)		PUT	/orders/:id/status	Update order status (Admin)		DELETE	/users/:id	Delete user (Admin)																					
Method	Endpoint	Description	Auth																																																		
POST	/payments	Create a payment for order																																																			
GET	/payments/:id	Get payment details by ID																																																			
Method	Endpoint	Description	Auth																																																		
GET	/users	Get all users (Admin)																																																			
PUT	/users/:id/role	Update user role (Admin)																																																			
PUT	/orders/:id/status	Update order status (Admin)																																																			
DELETE	/users/:id	Delete user (Admin)																																																			

8. Authentication

Authentication Flow:

1. User registers an account.

2. User logs in using email and password.
3. Backend verifies credentials.
4. JWT token is generated.
5. Token is stored in Local Storage.
6. Axios automatically sends the token in the Authorization header.
7. Protected routes verify the token before granting access.
8. Role-based authorization restricts access to Admin and Seller features.

Security measures include:

- Password hashing using bcrypt
- JWT Authentication
- Protected REST APIs
- Role-Based Access Control
- Environment Variables
- CORS Protection

9. User Interface:

Shopping Cart

My Cart

1 item



BRAND

Cactus Talkie

₹349

₹401

15% off

- 1 +

Remove

Save for later

Delivery by tomorrow | ₹40 delivery charges

Price Details

Price (1 item)	₹349
Discount	-₹17
Delivery Charges	₹40
Total Amount	₹372

You will save ₹17 on this order

Place Order

Enter delivery address & payment

Close

Complete your order with a shipping address and preferred payment method.

Full name

Phone

sri

Street address

City

State

Pincode

Country

India

Payment method

Cash On Delivery

Cancel

Confirm & Place Order

Order summary

Price	₹349
Discount	-₹17
Delivery	₹40
Total	₹372

You save ₹17 on this order.

My Orders

Order #bb92b1

Placed on 04/07/2025

Payment pending

Order status

Order total ₹250

Items

Smart Watch

Qty: 1

Shipping Address

101, New Market, Bangalore, India

04/07/2025

Order #bb92b1

Placed on 04/07/2025

Payment paid

Order status

Order total ₹250

Items

Smart Watch Pro

Qty: 1

Shipping Address

101, New Market, Bangalore, India

04/07/2025

Write Review

Submit Rating

Write your review

Submit Review

15% off



ELECTRONICS

Smart Watch Pro

4.0 1 reviews

20 In stock

Track fitness, messages and health metrics.

₹1899

15% OFF

You save ₹285

Color: Black Silver

Add to Cart Buy Now

Available offers

- Bank offer 10% instant discount
- Free delivery on orders above ₹500
- Easy returns and exchange

Customer ReviewsLatest

- 2104/07/2025good4

Share your reviewRating5 starsCommentTell us what you thinkSubmit review

Rating Breakdown

4.0 1 verified reviews

5+0%

4+100%

3+0%

2+0%

1+0%

ShopEZ
Empower. Shop. Save.

Search for products, brands and more

Search

SellerOrdersProfileWishlistCartLogout

Seller Dashboard

Manage your product listings and view sales activity.

My Products

13 active listings

Manage products

Average price	₹1661
Neem toys	₹250
Cactus Talkie	₹349
Geometry Box	₹201
Pens	₹500
story book	₹499
Ceiling Fan	₹3500
Kitchen Appliances	₹7000
Study Table	₹150
Office Chair	₹4000
Chair	₹900
Watche for Men	₹1500
Sneakers for Men	₹2000
Men's casual shirt	₹750

Product Management

Add or update products available through your seller/admin account.

Add new product

Title

Description

Category

Select category

Price

0

Discount (%)

0

Stock

1

Brand

Image URLs

Main image URL

Image URL 2

Second image URL

Image URL 3

Third image URL

Add product

New product

My Listings

13 products

Neem toys

₹250 • Stock: 23

EditDelete

Cactus Talkie

₹349 • Stock: 59

EditDelete

Geometry Box

₹201 • Stock: 30

EditDelete

Pens

₹500 • Stock: 52

EditDelete

story book

₹499 • Stock: 100

EditDelete

Ceiling Fan

₹3500 • Stock: 45

EditDelete

Kitchen Appliances

₹7000 • Stock: 24

EditDelete

Study Table

₹150 • Stock: 50

EditDelete

Office Chair

₹4000 • Stock: 15

EditDelete

Chair

₹900 • Stock: 12

EditDelete

Watche for Men

₹1500 • Stock: 32

EditDelete

Sneakers for Men

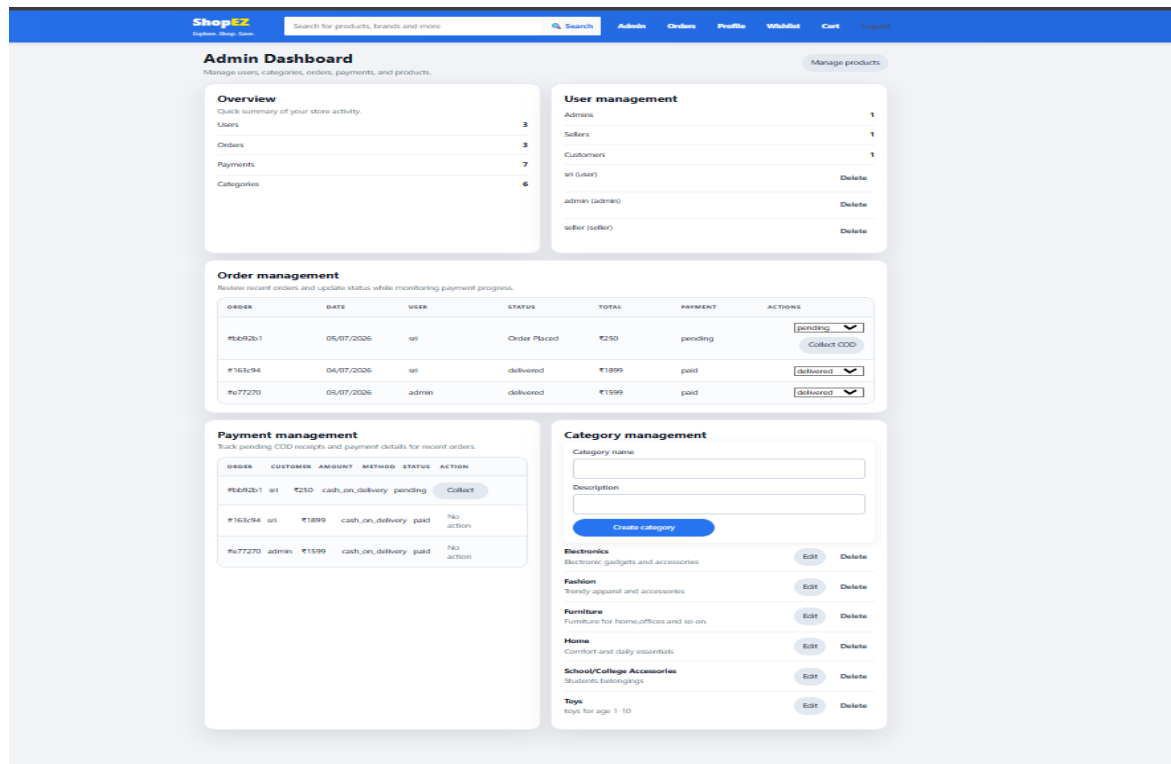
₹2000 • Stock: 25

EditDelete

Men's casual shirt

₹750 • Stock: 11

EditDelete



10. Testing

The project was tested using both manual and API testing methods.

Functional Testing

- Registration
- Login
- Product CRUD
- Search
- Filters
- Cart
- Wishlist
- Buy Now
- Orders
- Admin Features

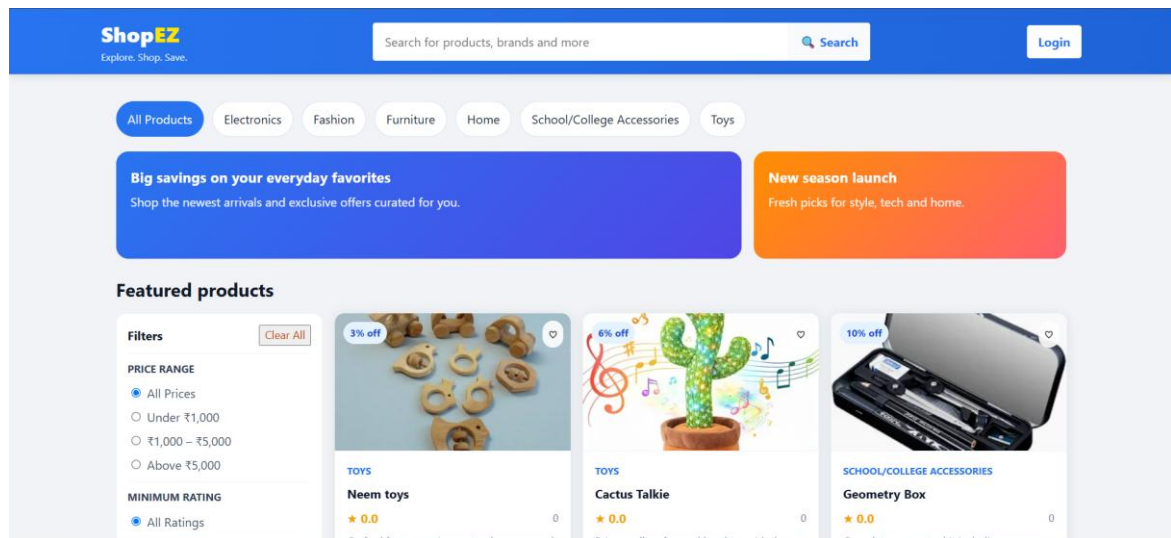
API Testing

- Postman

Browser Testing

- Google Chrome
- Microsoft Edge

11. Screenshots or Demo :



12. Known Issues :

Only Cash on Delivery (COD) payment is currently supported.

Product images are stored using image URLs instead of cloud storage.

Email notifications are not implemented.

13. Future Enhancements :

- Razorpay / Stripe Payment Integration
- Product Reviews and Ratings
- Email Verification
- Forgot Password
- Order Tracking
- Coupon & Discount System
- Product Recommendation System
- AI-Based Search
- Multi-language Support
- Cloud Deployment using Vercel and Render
- Sales Analytics Dashboard
- Push Notifications

- Invoice Generation
- Mobile Application Version